

Tests:

Pytest was used to test the app. The specific elements to be tested were chosen based on three main criteria.

- They must be testing something relevant to the app.
- They must be diverse, and cover a variety of elements.
- Ideally, they'd provide a useful learning experience.

The testing creates a database using the same structure that the one on the main app, so tests don't affect the real database.

Even though many tests may mock specific services, depending on the what's being tested, testing of redis requires a real use of redis. It was considered that tests that depend on external services would be separated from the main ones.

The default pytest execution for this project ignores such tests, and you have to specify it in the terminal to include them

```
(fastapi_env) C:\Users\borja\Desktop\fastAPI proj>python -m pytest -m external
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\borja\Desktop\fastAPI proj
configfile: pytest.ini
plugins: anyio-4.10.0, asyncio-1.4.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 8 items / 7 deselected / 1 selected

tests\external\test_cache.py .                                           [100%]

===== 1 passed, 7 deselected in 0.23s =====

(fastapi_env) C:\Users\borja\Desktop\fastAPI proj>python -m pytest
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\borja\Desktop\fastAPI proj
configfile: pytest.ini
plugins: anyio-4.10.0, asyncio-1.4.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 8 items / 1 deselected / 7 selected

tests\auth\test_auth.py ...                                           [ 42%]
tests\external\test_cache.py .                                         [ 57%]
tests\loans\test_loans.py ..                                          [ 85%]
tests\models\test_employee.py .                                       [100%]

===== 7 passed, 1 deselected in 0.65s =====
```

While in general I tried to keep the original code, some testing required slight modifications of the main app python files.

- Loan creation service committed the changes to the database. Since the service updates two tables and operation was needed to be atomic, it started like that. But flushing the changes to the database instead was a minimal change and it allowed easier testing
- To test loan creation due dates, the service accepted an object that is supposed to provide today's date. Testing was important enough to modify or even slightly complicate the method

Database:

Since the focus of the app was learning about cache and auth, the database was designed trying to keep fields and tables minimal, unless they were considered relevant to either functionality or the reality surrounding a hypothetical library service.

Books, borrowers and employees were necessary to manage the loans, and most of the database structure was added around these.

Another issue that was raised was how much data would be gathered per borrower/user. Since abusing a library system was not expected to be a huge issue, an email and phone was considered enough.

Cache

Cache was used mainly as a method to avoid making the same queries to the database many times over a short period of time.

Redis was chosen as the caching system, and if redis happens to be unavailable the app falls back to querying the database. A memory limit is established, and every element added has some “time to live” depending on:

- How expensive is to make the computations/query
- How relevant it is for it to stay updated
- What usage is an endpoint expected to have.

The endpoint that shows the top 5 loaned books in the last 7 days does not need constant updates, and it may have many requests, so a bigger TTL was chosen.

Also, for that same endpoint, I wondered what would happen for a heavy computation receiving multiple requests at the same time. In this case it was not needed, but I added a redis Lock to avoid repeating the same computations

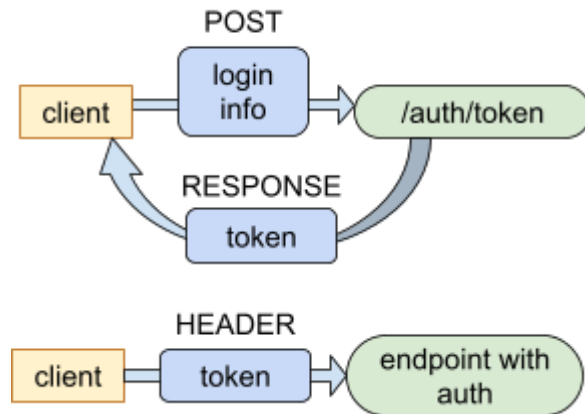
Endpoints where the content of the response needs for it to be updated , like checking availability of a book, do not use cache.

Auth

To restrict access to certain endpoints, JWT tokens were used for auth, each directly related to an employee. Existing packages make it easy to use jwt with FastAPI, and validation of a token is fast.

In this case, for the basic services the app has, if an employee is authenticated, it is also authorized.

The following diagram shows how successful authentication works.



A client sends its credentials to the “login” endpoint `/auth/token`, and if they are correct, a valid token is sent back in the response

The token can be reused later by the client, including it in the header of the http requests to endpoints that requires auth.

The signing and validation of tokens requires a secret key, which is stored in a file called `.env` within the project folder

The tokens expire a fixed time after their creation

Pydantic usage

Pydantic schemas were used to validate and represent some relevant inputs and outputs of endpoints. They were also used to help serialize elements to store them into Redis cache.

Full list of endpoints

auth

POST `/auth/token` Login

authors

GET `/authors/{id}` Get Author By Id

GET `/authors/` Get Authors By Name

books

GET `/books/book/{id}` Get Book By Id

GET `/books/available/{id}` Get Book Availability

GET `/books/search_title/` Get Books By Title

GET `/books/search_author/` Get Books By Author

GET `/books/top5` Get Top5 Recent Books

borrowers

GET `/borrowers/{borrower_id}` Get Borrowers By Id

POST `/borrowers/create` Create New Borrower Profile

loans

GET `/loans/active` Get Active Loans

POST `/loans/create` Create New Loan

PATCH `/loans/finish/{loan_id}` Finish Loan

Future improvements

The employee's model is very basic. It could be expanded in several ways.

- Another table related to employee's sensitive data
- Several types of employees could be added
- A "roles" table should be added to manage access to endpoints more selectively, including some kind of admin role.

Also, employees should be able to add book copies and books, and update physical copies status and loans freely.

There should be some kind of admin role, and endpoints modifying employees and accounts.

Some kind of control over incoming requests to prevent abuse would be ideal.

Queries done by public users are simple and a lot of options could be added to improve searching for books and authors.